Réalisation d'un mini-solveur de CSP binaires

Rapport de projet - Programmation Par Contraintes (MPRO)

RAKOTOBE Safidy VELTZ Siloé

17 octobre 2026

Résumé

Ce rapport présente un solveur générique de problèmes de satisfaction de contraintes (CSP) binaires à variables entières et domaines finis, écrit en Python. Le solveur combine un backtrack, trois niveaux de consistance (aucune, forward-checking, maintien de l'arc-consistance AC3), et des heuristiques de branchement interchangeables. Il est validé par comparaison croisée de toutes ses stratégies, puis évalué sur les n-Reines et le coloriage de graphe. Le point approfondi est l'*impact du cassage des symétries de couleurs* sur la détermination du nombre chromatique par dichotomie : nous comparons quatre méthodes (couleur imposée à un sommet, clique, règle dynamique d'interchangeabilité, combinaison) sur 15 instances DIMACS. Le cassage de symétrie réduit l'arbre de recherche de plus d'un ordre de grandeur sur les graphes de Mycielski et sur `queen6_6`, mais sur la majorité des instances c'est la *borne inférieure* donnée par une clique, plus que la symétrie, qui permet de conclure ; sur les instances faisables, le cassage est inutile et peut nuire.

Table des matières

1 Introduction **3**

2 Modélisation **3**

2.1 Représentation 3

2.2 Supports précalculés 4

2.3 Vérification indépendante 4

3 Résolution **4**

3.1 Backtrack 4

3.2 Gestion des domaines : le trail 5

3.3 Forward-checking 5

3.4 Arc-consistance : AC3 et MAC 6

3.5 Heuristiques de branchement 7

3.6 Options et statistiques 7

4 Validation **7**

5 Benchmarks **8**

5.1 n-Reines 8

5.2 Coloriage de graphe 10

6 Approfondissement : les symétries du coloriage **12**
6.1 Problème . . . 12
6.2 Méthodes comparées . . . 12
6.3 Protocole expérimental . . . 13
6.4 Résultats : preuve d'infaisabilité ($k = \chi - 1$) . . . 14
6.5 Résultats : recherche d'une solution ($k = \chi$) . . . 16
6.6 Conséquence pour la dichotomie . . . 16
6.7 Limites de l'étude . . . 17
6.8 Conclusion de l'approfondissement . . . 17

7 Bilan **18**
7.1 Points forts . . . 18
7.2 Points faibles . . . 18
7.3 Perspectives . . . 18

8 Conclusion **18**

A Utilisation et reproduction **19**

B Description des fichiers **19**

1 Introduction

L'objectif du projet est d'implémenter un solveur générique de CSP dont les contraintes sont binaires et les variables entières, avec des domaines finis. Le travail demandé comporte quatre volets : (i) un *modèle* de CSP, (ii) un algorithme de *backtrack* dont les heuristiques de branchement sont modifiables, (iii) des algorithmes de *consistance* (arc-consistance AC3 ou AC4 et forward-checking) activables à la demande, (iv) des *benchmarks* (n-Reines, coloriage, autres problèmes) et un *approfondissement* sur un point au choix.

Contributions.

- Un modèle de CSP binaire en extension, avec génération automatique des contraintes symétriques et supports précalculés (section 2).
- Un moteur de recherche avec trois niveaux de consistance, cinq heuristiques de variable, trois heuristiques de valeur, un mécanisme de *trail* pour la restauration des domaines, un timeout et des statistiques détaillées (section 3).
- Une validation par comparaison croisée de 45 stratégies (section 4).
- Des benchmarks sur les n-Reines et sur le coloriage, avec calcul du nombre chromatique par dichotomie (section 5).
- Une étude expérimentale des symétries de couleurs (section 6).

Organisation du code. Le solveur est découpé en modules qui ne dépendent d'aucune instance : modèle (`csp.py`), recherche, consistance et heuristiques (`solveur.py`), générateurs de problèmes et optimisation du coloriage (`problemes.py`), plus les scripts de benchmark, d'étude des symétries et de tests (annexe B).

2 Modélisation

Toutes les contraintes sont considérées de manière *directionnelle*, comme dans le cours : une contrainte entre x et y génère à la fois C_{xy} et $C_{yx} = \{(b, a) : (a, b) \in C_{xy}\}$.

2.1 Représentation

Le format (liste de variables, dictionnaire de domaines, dictionnaire de contraintes dont les valeurs sont des ensembles de tuples autorisés) est encapsulé dans une classe `CSP` (fichier `csp.py`) :

```
csp = CSP()
csp.ajouter_variable(0, [0, 1])
csp.ajouter_variable(1, [0, 1])
csp.ajouter_contrainte(0, 1, {(0, 1), (1, 0)}) # en extension
csp.ajouter_contrainte_intension(0, 1, lambda a, b: a != b) # en intension
```

Symétrisation automatique. Si l'utilisateur déclare seulement C_{xy} et pas C_{yx}, un test d'affectation qui ne regarde qu'un sens ignore la contrainte selon l'ordre dans lequel les variables sont affectées, et le solveur renvoie alors de fausses solutions. La méthode `ajouter_contrainte` génère donc toujours les deux sens, et l'utilisateur ne décrit chaque contrainte qu'une fois (ce qui divise par deux le code de construction, par exemple pour les n-Reines).

Conjonction. Si plusieurs contraintes portent sur la même paire de variables, on retient l'*intersection* de leurs ensembles de tuples, ce qui correspond à leur conjonction. L'intersection est calculée dans un nouvel objet : les ensembles de tuples partagés entre plusieurs arêtes (cas du coloriage, où la relation $\neq$ est identique pour toutes les arêtes) ne sont jamais modifiés.

Contraintes en intension. Un prédicat $p(a, b)$ est converti en extension à la construction. Cela garde un seul format interne (celui que les solveurs de PPC utilisent par défaut) tout en laissant l'utilisateur écrire $x \neq y$ directement.

2.2 Supports précalculés

Les algorithmes de consistance posent sans cesse la question : « la valeur $a \in D_x$ a-t-elle un support dans D_y ? ». Parcourir les tuples de C_{xy} à chaque fois serait coûteux. À la création du solveur, on compile donc, pour chaque arc (x, y) et chaque valeur $a \in D_x$:

$$S_{xy}(a) = \{\, b \in D_y : (a, b) \in C_{xy} \,\},$$

stocké comme `frozenset`. La question devient « $S_{xy}(a) \cap D_y^{\text{courant}} \neq \emptyset$? », réalisée par `isdisjoint` en $O(\min(|S_{xy}(a)|, |D_y|))$ au pire, et le plus souvent très vite. Le coût mémoire est proportionnel au nombre de tuples. Le calcul préalable est linéaire en ce nombre.

2.3 Vérification indépendante

La méthode `CSP.est_solution` contrôle une instanciation directement sur les tuples (domaines et contraintes), sans passer par aucun code de recherche. Elle sert dans les tests et dans les benchmarks (chaque solution renvoyée est vérifiée).

3 Résolution

3.1 Backtrack

L'algorithme 1 est le backtrack du cours, instancié avec les trois points de variation : le niveau de consistance, le choix de la variable et l'ordre des valeurs.

Algorithme 1 Backtrack avec consistance et heuristiques

Entrée : instanciation partielle I, domaines courants D
Sortie : VRAI si I s'étend en solution (la solution est mémorisée)

```
1 : si I est complète alors
2 :     retourner VRAI
3 : fin si
4 : x ← ChoisirVariable(variables libres, D)
5 : pour v ∈ OrdonnerValeurs(x, D) faire
6 :     si consistance = none et v incompatible avec I alors
7 :         continuer
8 :     fin si
9 :     marque ← taille du trail
10 :    I ← I ∪ {x ↦ v} ; réduire D_x à {v}
11 :    ok ← VRAI
12 :    si consistance = fc alors
13 :        ok ← ForwardChecking(x)
14 :    sinon si consistance = mac alors
15 :        ok ← AC3({(y, x) : y voisin libre de x})
16 :    fin si
17 :    si ok et Backtrack(I) alors
18 :        retourner VRAI
19 :    fin si
20 :    restaurer les domaines jusqu'à marque ; I ← I \ {x}
21 : fin pour
22 : retourner FAUX
```

En mode `none` (backtrack pur), la compatibilité d'une valeur avec les variables déjà affectées est testée par les supports : $\forall y$ voisin de x déjà instancié, $I(y) \in S_{xy}(v)$. Avec FC ou MAC ce test est inutile, puisque le filtrage a déjà retiré les valeurs incompatibles des domaines des variables libres.

Variantes de recherche. Le même moteur permet de chercher la première solution, de les énumérer toutes (option `toutes_les_solutions`, utilisée pour la validation) ou de s'arrêter après un nombre donné.

3.2 Gestion des domaines : le trail

Copier tous les domaines à chaque nœud serait coûteux (cours 3, gestion de la mémoire). Nous représentons les domaines courants par des ensembles, auxquels s'ajoute une pile, le *trail*, qui enregistre chaque couple (x, a) retiré. Une *marque* est la taille du trail avant une décision ; restaurer consiste à dépiler jusqu'à cette marque en réinsérant les valeurs.

Fixer x à v revient à retirer toutes les autres valeurs de D_x (donc à les empiler sur le trail) : une variable affectée est une variable à domaine singleton. Ce choix uniformise le traitement : les algorithmes de consistance n'ont pas à distinguer variables libres et affectées, et un retour arrière annule en un seul geste l'affectation *et* tout le filtrage qu'elle a provoqué.

3.3 Forward-checking

Dès qu'une variable x est affectée à a, on filtre les domaines des voisins libres sans propager plus loin (algorithme 2).

Algorithme 2 Forward-checking après l'affectation $x \mapsto a$

pour tout y voisin libre de x **faire**
    retirer de D_y toute valeur $b \notin S_{xy}(a)$
    si $D_y = \emptyset$ **alors**
        retourner FAUX
    fin si
fin pour
retourner VRAI

Le coût d'un appel est $O(\deg(x) \cdot d)$; il est faible et ne dépend pas de la taille de l'arbre. FC ne détecte une impasse que lorsqu'un domaine *voisin direct* d'une variable affectée est vidé.

3.4 Arc-consistance : AC3 et MAC

Une valeur $a \in D_x$ est *arc-consistante* si, pour chaque voisin y, elle a un support dans D_y. Un CSP est arc-consistant si toutes ses valeurs le sont. Retirer une valeur sans support ne change pas l'ensemble des solutions (elle ne peut appartenir à aucune solution). L'algorithme 3 est l'AC3 du cours.

Algorithme 3 AC3 (file d'arcs, sans doublons dans la file)

Entrée : une liste d'arcs initiaux Q
tant que $Q \neq \emptyset$ **faire**
    retirer (x, y) de Q
    si RÉVISER(x, y) **alors** ▷ retire de D_x les valeurs sans support dans D_y
        si $D_x = \emptyset$ **alors**
            retourner FAUX
        fin si
        pour tout z voisin de x, $z \neq y$ **faire**
            ajouter (z, x) à Q s'il n'y est pas déjà
        fin pour
    fin si
fin tant que
retourner VRAI

Quand D_x est modifié, on ne remet dans la file que les arcs (z, x) avec $z \neq y$: les valeurs retirées de D_x n'avaient aucun support dans D_y ; la contrainte étant symétrique, aucune valeur de D_y ne s'appuyait sur elles. Ensuite, un ensemble des arcs présents évite les doublons dans la file.

Complexité. En notant e le nombre de contraintes et d la taille maximale des domaines, chaque arc peut être remis dans la file au plus d fois (à chaque retrait d'une valeur du domaine du voisin), et une révision coûte $O(d^2)$ au pire : la complexité temporelle au pire est $O(e\, d^3)$, celle d'AC3. Le précalcul des supports rend la révision beaucoup plus rapide en pratique.

MAC (*Maintaining Arc-Consistency*) applique AC3 après chaque affectation $x \mapsto a$, avec pour file initiale les arcs (y, x) pour tout voisin libre y. Les domaines étant ramenés à un singleton pour x, la première passe de MAC contient le filtrage de FC ; la propagation en cascade qui suit retire donc au moins autant de valeurs que FC. Une option permet aussi d'appliquer AC3 *une seule fois à la racine* (avant toute décision), quel que soit le niveau de consistance choisi ensuite.

Nous n'avons pas implémenté AC4 : AC3 est plus simple, déjà suffisant pour nos tailles d'instances, et le temps du projet a été consacré à l'approfondissement.

3.5 Heuristiques de branchement

Les heuristiques sont des fonctions interchangeables passées au solveur ; en cas d'égalité on retient la première variable dans l'ordre de déclaration, ce qui rend les exécutions déterministes.

Nom	**Variable choisie parmi les variables libres**
`static`	la première dans l'ordre de déclaration
`dom`	plus petit domaine courant (*first-fail*)
`deg`	plus grand nombre de voisins libres
`dom_deg`	minimise $\lvert D_x\rvert / \deg(x)$ (deg = nombre de voisins libres)
`dom_wdeg`	minimise $\lvert D_x\rvert/\mathrm{wdeg}(x)$, où $\mathrm{wdeg}(x) = \sum_{y \text{ libre}} w_{xy}$

Tab. 1 : Heuristiques de choix de variable.

Dans `dom_wdeg`, chaque contrainte porte un poids w_{xy} initialisé à 1 et incrémenté chaque fois qu'elle provoque un *vidage de domaine* (en FC ou en AC3). Les variables liées par des contraintes « difficiles » sont ainsi privilégiées. Sans consistance (mode `none`), aucun vidage n'est détecté, donc les poids restent à 1.

Pour les valeurs : `static` (ordre croissant), `random` (ordre aléatoire à graine fixée) et `lcv` (*least constraining value*), qui trie les valeurs par nombre croissant de valeurs retirées chez les voisins libres, c'est-à-dire $\sum_y \big(|D_y| - |S_{xy}(v) \cap D_y|\big)$.

3.6 Options et statistiques

Le solveur expose les statistiques suivantes, utilisées dans tous les benchmarks :

- **nœuds** : affectations tentées (cohérentes avec le passé en mode `none` ; cohérentes avec les domaines courants sinon) ;
- **retours arrière** : nœuds de l'arbre dont aucune valeur ne mène à une solution ;
- **révisions** : nombre de filtrages élémentaires (un par voisin en FC, un par appel à Réviser en MAC) : c'est la mesure du *coût de la consistance* ;
- **temps** et **timeout**. En cas de dépassement du temps imparti (option `timeout`), un drapeau l'indique : on peut ainsi distinguer « pas de solution » de « inconnu », distinction indispensable pour la dichotomie du coloriage.

4 Validation

Le principe est que *toutes les stratégies doivent trouver exactement les mêmes solutions* : une divergence signale un bug de propagation ou de restauration des domaines. Les tests (`tests.py`) comprennent :

1. **Modèle.** Symétrisation des contraintes, vérificateur de solutions.
2. **n-Reines, comptage.** Pour $n \in \{4, 5, 6, 7\}$: les 45 combinaisons (3 consistances $\times$ 5 heuristiques de variable $\times$ 3 de valeur) énumèrent exactement le nombre connu de solutions

($n = 4 \mapsto 2$, $5 \mapsto 10$, $6 \mapsto 4$, $7 \mapsto 40$). Pour $n = 8$ (92 solutions), trois combinaisons représentatives sont testées. Chaque solution est contrôlée par le vérificateur indépendant.

3. **Infaisabilité.** Le triangle n'est pas 2-colorable : toutes les stratégies répondent « pas de solution », sans timeout.
4. **Symétries de couleurs.** Pour 26 graphes (Petersen et 25 graphes aléatoires de 9 à 14 sommets), les 5 modes de cassage de symétrie, 3 consistances et 3 heuristiques de variable : $\chi-1$ couleurs sont toujours insuffisantes et χ suffisent, partout, et chaque solution est valide dans le modèle *sans* cassage (section 6).
5. **Timeout.** Une recherche volontairement trop longue (40-Reines en backtrack pur) est interrompue.

5 Benchmarks

Les temps sont mesurés avec un interpréteur Python ; ils dépendent de la machine (et de sa charge au moment de l'exécution) et n'ont de valeur que par leurs ordres de grandeur. Le nombre de nœuds et de révisions, lui, est indépendant de la machine, tant que le timeout n'est pas atteint.

5.1 n-Reines

Modèle. Une variable par ligne $i \in \{0, \ldots, n-1\}$ dont la valeur est la colonne de la reine ; pour $i < j$, la contrainte binaire autorise (a, b) ssi $a \neq b$ (pas de colonne commune) et $|a - b| \neq j - i$ (pas de diagonale commune). La construction en extension est en $O(n^4)$ au total.

Résultats. Le tableau 2 donne la première solution trouvée, avec la valeur `static` pour l'ordre des valeurs et un timeout de 10 s.

n	Consistance	Variable	Nœuds	Retours	Révisions	Temps (s)
8	aucune	static	113	105	0	0,001
8	FC	static	88	45	253	0,002
8	FC	dom	75	44	230	0,001
8	MAC	static	20	2	707	0,004
8	MAC	dom	20	2	707	0,003
8	MAC	dom_wdeg	22	3	663	0,002
12	aucune	static	261	249	0	0,011
12	FC	static	193	99	723	0,002
12	FC	dom	153	95	642	0,006
12	MAC	static	41	6	2 640	0,008
12	MAC	dom	51	13	2 971	0,012
12	MAC	dom_wdeg	55	15	3 035	0,015
16	aucune	static	10 052	10 036	0	0,407
16	FC	static	7 560	4 362	32 969	0,091
16	FC	dom	44	21	247	0,001
16	MAC	static	1 403	460	122 859	0,443
16	MAC	dom	21	1	2 315	0,011
16	MAC	dom_wdeg	21	1	2 306	0,017
20	aucune	static	199 635	199 615	0	$\approx 7{,}7$
20	FC	static	145 151	84 380	722 627	1,645
20	FC	dom	145	92	876	0,002
20	MAC	static	25 925	9 014	2 982 058	6,795
20	MAC	dom	52	13	7 332	0,022
20	MAC	dom_wdeg	50	13	6 876	0,019

Tab. 2 : n-Reines, première solution (ordre des valeurs : croissant). Pour $n = 20$ en backtrack pur, la recherche termine avant le timeout de 10 s (environ 7,7 s sur notre machine ; le nombre de nœuds ne dépend pas de la machine).

Analyse.

1. *La consistance réduit l'arbre.* Pour $n = 16$ avec l'ordre statique, passer du backtrack pur à FC ramène les nœuds de 10 052 à 7 560, puis MAC à 1 403. Plus le raisonnement est puissant, plus on visite peu de nœuds. Pour $n = 20$, le backtrack pur visite 199 635 nœuds, contre 145 151 pour FC et 25 925 pour MAC.

2. *L'heuristique de variable pèse davantage que la consistance.* Avec `dom`, FC ne visite que 145 nœuds pour $n = 20$, contre 145 151 avec l'ordre statique : un facteur 1 000 environ, très supérieur au gain apporté par le passage de FC à MAC en ordre statique (facteur 5,6). Le principe du cours (« aller le plus vite vers une contradiction ») est confirmé.

3. *Moins de nœuds n'implique pas moins de temps.* Avec l'ordre statique pour $n = 20$, MAC visite 5,6 fois moins de nœuds que FC mais effectue près de 3 millions de révisions (contre 0,72 million) et se révèle plus lent (6,8 s contre 1,6 s). Quand une heuristique performante rend l'arbre minuscule (`dom`), la consistance supplémentaire n'a plus rien à gagner et coûte son prix : FC est alors environ 11 fois plus rapide que MAC (0,002 s contre 0,022 s). La conclusion du cours (« en pratique, souvent le forward-checking ») est retrouvée.

4. *`dom_wdeg` ne se distingue pas ici.* Les n-Reines sont très symétriques et sans structure de « contraintes difficiles » : les poids n'apportent rien (résultats quasi identiques à `dom`).

5.2 Coloriage de graphe

Modèle. Étant donnés un graphe $G = (V, E)$ et un entier k, le CSP *décision* associe à chaque sommet une variable de domaine $\{0, \dots, k-1\}$ (une couleur) et à chaque arête $\{u, v\}$ la contrainte $u \neq v$. Les instances sont lues au format DIMACS (`.col`) par `lire_col`.

Optimisation par dichotomie. Le coloriage est un problème d'optimisation, alors que la PPC traite des problèmes de décision. Comme au cours 1, nous résolvons une suite de problèmes de décision par dichotomie (algorithme 4) :

- la *borne supérieure* ub est le nombre de couleurs d'un coloriage glouton (Welsh–Powell : sommets triés par degré décroissant) ;
- la *borne inférieure* lb est la taille d'une clique trouvée par une heuristique gloutonne (depuis chacun des 100 sommets de plus fort degré, on ajoute à chaque pas le candidat qui a le plus de voisins parmi les candidats) ;
- chaque décision « G est-il k-colorable ? » a son propre timeout.

Algorithme 4 Nombre chromatique par dichotomie

$ub \leftarrow$ coloriage glouton ; $lb \leftarrow |$clique gloutonne$|$
tant que $lb < ub$ **faire**
  $k \leftarrow \lfloor (lb + ub)/2 \rfloor$
  résoudre le CSP « G est k-colorable ? » avec timeout
  si solution σ trouvée **alors**
    $ub \leftarrow |\sigma(V)|$ ▷ nombre de couleurs effectivement utilisées
  sinon si timeout **alors**
    arrêter ▷ $lb \leq \chi \leq ub$ non prouvé
  sinon
    $lb \leftarrow k + 1$
  fin si
fin tant que
retourner (lb, ub) ▷ $\chi(G) = lb = ub$ si prouvé

Ce sont les appels *infaisables* (qui font monter lb) qui sont chers : le solveur doit alors parcourir l'arbre entier. Nous y reviendrons à la section 6.

Résultats. Le tableau 3 donne le résultat de la dichotomie complète sur cinq instances, *sans* cassage de symétrie (`aucune`) et *avec* le réglage par défaut `clique+dyn`, avec un timeout de 10 s par décision. Le temps est la somme des temps des appels de décision. Les autres instances sont étudiées au niveau des problèmes de décision à la section 6.

Instance	Conf.	Variable	aucune			clique+dyn	
			lb–*ub*	Nœuds	Temps (s)	Nœuds	Temps (s)
`myciel3`	FC	dom	4–4	141	0,001	26	0,000
($\|V\|$=11, $\|E\|$=20)	MAC	dom	4–4	51	0,001	9	0,000
	MAC	dom_wdeg	4–4	37	0,001	9	0,000
`myciel4`	FC	dom	5–5	31 129	0,207	1 016	0,011
($\|V\|$=23, $\|E\|$=71)	MAC	dom	5–5	10 111	0,510	340	0,016
	MAC	dom_wdeg	5–5	3 825	0,226	126	0,010
`queen5_5`	FC	dom	5–5	25	0,000	25	0,000
($\|V\|$=25, $\|E\|$=160)	MAC	dom	5–5	25	0,001	25	0,001
	MAC	dom_wdeg	5–5	25	0,008	25	0,002
`queen6_6`	FC	dom	7–7	181 975	4,125	3 069	0,055
($\|V\|$=36, $\|E\|$=290)	MAC	dom	6–7 †	46 299	10,093	902	0,138
	MAC	dom_wdeg	6–7 †	41 591	10,370	396	0,077
`queen7_7`	FC	dom	7–7	1 095	0,016	14 662	0,296
($\|V\|$=49, $\|E\|$=476)	MAC	dom	7–7	434	0,088	4 510	0,915
	MAC	dom_wdeg	7–7	6 443	1,845	679	0,182

Tab. 3 : Nombre chromatique par dichotomie, sans cassage de symétrie (`aucune`) et avec `clique+dyn`. Avec `clique+dyn`, toutes les exécutions prouvent $lb = ub = \chi$. † : timeout de 10 s atteint sur la décision $k = 6$; seul l'encadrement $6 \leq \chi \leq 7$ est établi.

Effet du cassage de symétrie.

- *`myciel4`.* Le cassage divise le nombre de nœuds par 31 avec FC (31 129 → 1 016), par 30 avec MAC (10 111 → 340) et par 30 avec `dom_wdeg` (3 825 → 126). Le temps suit : de 0,207 s à 0,011 s pour FC, de 0,510 s à 0,016 s pour MAC.
- *`queen6_6`.* C'est le cas le plus net. Sans cassage, seul FC conclut (181 975 nœuds, 4,1 s) ; MAC avec `dom` comme avec `dom_wdeg` atteignent le timeout de 10 s sur la décision $k = 6$ et n'établissent que $6 \leq \chi \leq 7$. Avec `clique+dyn`, les trois configurations concluent en moins de 0,14 s : 3 069 nœuds pour FC (59 fois moins, 75 fois plus rapide), 902 pour MAC, 396 pour `dom_wdeg`. Le cassage fait donc passer deux des trois configurations de « non résolu » à « résolu ».
- *`myciel3` et `queen5_5`.* Instances trop petites pour que l'effet se voie en temps (pour `queen5_5`, la clique a 5 sommets et $\chi = 5$: la dichotomie ne fait que construire un coloriage, 25 nœuds pour 25 sommets, sans retour arrière).
- *`queen7_7` : un cas où le cassage nuit.* Ici $lb = \omega = 7 = \chi$ dès le départ et il suffit de trouver un coloriage. Le cassage *augmente* le nombre de nœuds : 1 095 → 14 662 avec FC (facteur 13) et 434 → 4 510 avec MAC (facteur 10), et le temps de 0,016 s à 0,296 s pour FC. C'est le phénomène décrit à la section 6 : imposer une clique sans rapport avec l'heuristique de branchement dévie la recherche d'une solution. (Les nœuds de ce tableau ne se comparent pas directement à ceux du tableau 6, qui concerne un seul appel $k = \chi$.)

Effet de `dom_wdeg`. Avec le cassage de symétrie, `dom_wdeg` réduit systématiquement le nombre de nœuds de MAC par rapport à `dom`, et d'autant plus que l'instance est difficile : facteur 2,7 sur `myciel4` (126 contre 340), 2,3 sur `queen6_6` (396 contre 902) et 6,6 sur `queen7_7` (679 contre 4 510). Le gain en temps suit quand l'arbre est assez gros pour amortir le coût du calcul des poids : 0,182 s contre 0,915 s sur `queen7_7` (5 fois plus rapide), 0,077 s contre 0,138 s sur `queen6_6`, 0,010 s contre 0,016 s sur `myciel4`. Sans cassage de symétrie, le comportement est

moins régulier : `dom_wdeg` aide sur `myciel4` (3 825 nœuds contre 10 111, 0,226 s contre 0,510 s) mais pénalise `queen7_7` (6 443 nœuds contre 434 ; 1,8 s contre 0,09 s), où l'ordre de `dom` trouvait presque directement un coloriage. Ces mesures sont à interpréter avec prudence : une seule exécution par configuration, et `dom_wdeg` dépend de l'historique des échecs. L'effet du cassage de symétrie, comparé à tous les autres modes, est l'objet de la section suivante.

6 Approfondissement : les symétries du coloriage

6.1 Problème

Dans le modèle précédent, les couleurs sont *interchangeables* : si σ est un k-coloriage et π une permutation de $\{0, \dots, k-1\}$, alors $\pi \circ \sigma$ en est un autre. Une solution engendre ainsi $k!$ solutions équivalentes. Pour un problème *faisable*, ces copies ne gênent guère puisqu'une seule suffit. Pour un problème *infaisable*, c'est dramatique : le backtrack doit échouer séparément sur chacune des copies équivalentes d'une même impasse. Or la dichotomie du coloriage a précisément besoin de prouver que $k-1$ couleurs ne suffisent pas, pour établir la borne inférieure. Nous étudions donc comment limiter ces symétries et leur impact sur la taille de l'arbre et sur le temps de calcul.

6.2 Méthodes comparées

`aucune`. Modèle de base.

`sommet`. On impose la couleur 0 au sommet de plus fort degré. C'est valide : dans tout k-coloriage, on peut permuter les couleurs pour que ce sommet reçoive la couleur 0.

`clique`. On calcule une clique $K = \{s_0, \dots, s_{m-1}\}$ par l'heuristique gloutonne et on impose $\sigma(s_i) = i$. Les sommets d'une clique ont forcément des couleurs deux à deux distinctes ; en permutant, on peut toujours leur donner les couleurs $0, \dots, m-1$. Si $m > k$, le domaine de s_k devient vide : l'infaisabilité est immédiate. Cette méthode a donc un double rôle : casser des symétries, *et* fournir une borne inférieure ($\chi \geq m$).

`dynamique`. À chaque nœud, parmi les valeurs candidates de la variable choisie, on n'essaie qu'*une valeur par classe de couleurs interchangeables*. Deux couleurs c, c' sont déclarées interchangeables dans l'état courant si elles figurent dans exactement les mêmes domaines :

$$\{y \in V : c \in D_y\} = \{y \in V : c' \in D_y\}.$$

Cette « signature » est calculée sur toutes les variables, y compris les variables déjà affectées (domaines singletons) et celles dont le domaine a été restreint par une clique ou par la propagation ; une couleur déjà utilisée a donc une signature qui lui est propre. L'algorithme est donné ci-dessous.

Algorithme 5 Filtre dynamique des couleurs interchangeables

Entrée : variable x, liste ordonnée de valeurs candidates L
1 : $vus \leftarrow \emptyset$; $R \leftarrow$ liste vide
2 : **pour tout** $c \in L$ dans l'ordre **faire**
3 : $\quad s \leftarrow \{y \in V : c \in D_y\}$
4 : $\quad$ **si** $s \notin vus$ **alors**
5 : $\quad\quad vus \leftarrow vus \cup \{s\}$; ajouter c à R
6 : $\quad$ **fin si**
7 : **fin pour**
8 : **retourner** R

Proposition 1 (Validité de la règle dynamique)**.** *Soit un état de la recherche (domaines courants D, éventuellement filtrés de manière valide) et c, c' deux couleurs de signatures égales. Si une solution de l'état affecte x à c', il en existe une qui affecte x à c, et inversement. Par conséquent, ne tenter qu'un représentant par classe ne fait perdre ni solution, ni preuve d'infaisabilité.*

Démonstration. Soit τ la transposition de c et c' et σ une solution de l'état. Posons $\sigma' = \tau \circ \sigma$. Pour tout sommet y, comme $c \in D_y \Leftrightarrow c' \in D_y$, l'ensemble D_y est stable par τ, donc $\sigma'(y) = \tau(\sigma(y)) \in D_y$. Pour toute arête $\{u, v\}$, $\sigma(u) \neq \sigma(v)$ et τ est injective, donc $\sigma'(u) \neq \sigma'(v)$. Ainsi σ' est une solution, et $\sigma(x) = c' \Rightarrow \sigma'(x) = c$. □

La preuve ne suppose rien sur le niveau de consistance : seul compte que les domaines courants soient des restrictions valides (qui ne perdent pas de solutions), ce qui est le cas pour FC et MAC. Elle ne suppose pas non plus de clique. Elle s'applique aussi à la racine : toutes les couleurs ont la même signature, donc seule la couleur 0 est essayée pour la première variable, ce qui revient à fixer à 0 la variable choisie par l'heuristique (comme `sommet`, mais sans choix particulier du sommet).

`clique+dyn`. Les deux précédentes combinées.

Validation de la correction. Un cassage de symétrie ne doit *jamais* changer la satisfiabilité. Nous l'avons vérifié sur 26 graphes, avec les 5 modes, les 3 consistances et 3 heuristiques de variable : pour chaque graphe, $\chi - 1$ couleurs sont toujours déclarées insuffisantes, χ suffisent, et toute solution renvoyée est valide dans le modèle sans cassage. Seul le *nombre* de solutions énumérées diminue, ce qui est voulu.

6.3 Protocole expérimental

Pour chaque instance de nombre chromatique χ connu, nous résolvons deux problèmes de décision :

- $k = \chi - 1$: *infaisable*, l'arbre doit être parcouru intégralement ;
- $k = \chi$: *faisable*, il suffit de trouver un coloriage.

Chaque problème est résolu avec FC et MAC (heuristique de variable `dom`, valeurs croissantes), pour chacun des cinq modes, avec un timeout de 3 s. Les 15 instances DIMACS sont : `anna`, `david`, `games120`, `huck`, `jean`, `miles250`, `miles500`, `myciel3` à `myciel6` et `queen5_5` à `queen8_8` (script `etude_symetries.py`, résultats bruts dans `resultats_symetries.csv`). Pour chaque exécution on relève le résultat, le nombre de nœuds et le temps. Dans les tableaux, TO signifie que le timeout a été atteint : le problème n'est alors *pas résolu*.

6.4 Résultats : preuve d'infaisabilité ($k = \chi - 1$)

Le tableau 4 donne, pour chaque instance, le nombre de nœuds nécessaires pour prouver que $\chi - 1$ couleurs ne suffisent pas (ω est la taille de la clique trouvée).

Instance, k	Conf.	`aucune`	`sommet`	`clique`	`dynamique`	`clique+dyn`
`anna` (ω=11), k=10	FC	TO	TO	0	TO	0
	MAC	TO	TO	0	TO	0
`david` (ω=11), k=10	FC	TO	TO	0	232	0
	MAC	TO	TO	0	168	0
`games120` (ω=9), k=8	FC	TO	TO	0	TO	0
	MAC	TO	TO	0	TO	0
`huck` (ω=11), k=10	FC	TO	TO	0	TO	0
	MAC	TO	TO	0	TO	0
`jean` (ω=10), k=9	FC	TO	TO	0	TO	0
	MAC	TO	TO	0	TO	0
`miles250` (ω=8), k=7	FC	TO	1 957	0	TO	0
	MAC	TO	1 237	0	TO	0
`miles500` (ω=20), k=19	FC	TO	TO	0	TO	0
	MAC	TO	TO	0	TO	0
`myciel3` (ω=2), k=3	FC	141	51	26	24	26
	MAC	51	17	9	9	9
`myciel4` (ω=2), k=4	FC	30 976	5 890	1 964	1 293	984
	MAC	10 048	1 960	654	421	329
`myciel5` (ω=2), k=5	FC	TO	TO	TO	TO	TO
	MAC	TO	TO	TO	TO	TO
`myciel6` (ω=2), k=6	FC	TO	TO	TO	TO	TO
	MAC	TO	TO	TO	TO	TO
`queen5_5` (ω=5), k=4	FC	64	22	0	4	0
	MAC	40	10	0	3	0
`queen6_6` (ω=6), k=6	FC	TO	33 266	251	254	251
	MAC	TO	9 266	79	75	79
`queen7_7` (ω=7), k=6	FC	1 956	986	0	6	0
	MAC	1 236	626	0	5	0
`queen8_8` (ω=8), k=8	FC	TO	TO	TO	TO	TO
	MAC	TO	TO	TO	TO	TO

Tab. 4 : Preuve d'infaisabilité ($k = \chi - 1$) : nombre de nœuds (TO = timeout de 3 s, problème non résolu).

Le tableau 5 résume le nombre d'instances sur 15 dont l'infaisabilité est prouvée en moins de 3 s.

	`aucune`	`sommet`	`clique`	`dynamique`	`clique+dyn`
FC	4	6	12	6	12
MAC	4	6	12	6	12

Tab. 5 : Nombre d'instances (sur 15) dont l'infaisabilité à $k = \chi - 1$ est prouvée en 3 s.

La figure 1 visualise la réduction de l'arbre avec FC sur les deux instances où tous les modes (sauf éventuellement `aucune`) terminent.

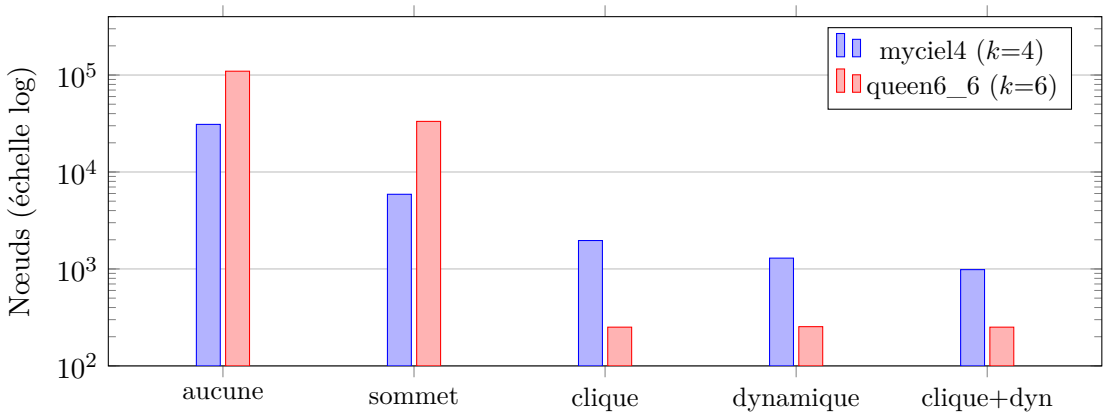


Fig. 1 : Nombre de nœuds pour prouver l'infaisabilité avec FC et `dom`. Pour `queen6_6`, la barre `aucune` est un *minorant* : le timeout de 3 s est atteint après 109 529 nœuds sans conclure (avec un timeout de 10 s, la preuve aboutit en 181 975 nœuds, tableau 3).

Lecture des résultats.

1. **Un gain massif quand la symétrie est le vrai obstacle.** Sur `myciel4`, le nombre de nœuds est divisé par 31 avec FC (30 976 → 984) comme avec MAC (10 048 → 329). Sur `queen6_6`, le modèle de base ne termine pas en 3 s (plus de 109 529 nœuds avec FC, plus de 9 159 avec MAC), alors que le cassage de symétrie conclut en 251 nœuds avec FC (0,005 s) et 79 avec MAC (0,019 s) : un facteur d'au moins 400 en nœuds. Le mode `sommet` seul suffit à terminer, mais plus lentement (FC : 33 266 nœuds, 1,1 s ; MAC : 9 266 nœuds, 2,7 s).

2. **Sur la majorité des instances, c'est la borne inférieure, pas la symétrie, qui conclut.** Pour 9 instances sur 15 (`anna`, `david`, `games120`, `huck`, `jean`, `miles250`, `miles500`, `queen5_5`, `queen7_7`), la clique trouvée a déjà $\omega = \chi$ sommets, donc $\omega > k = \chi - 1$: le mode `clique` déclare l'infaisabilité en 0 nœud, simplement parce qu'une clique de ω sommets ne peut pas être colorée avec moins de ω couleurs. C'est un effet de *borne inférieure*, qui n'a rien à voir avec les symétries. Sur les six instances réelles à grosse clique (`anna`, `games120`, `huck`, `jean`, `miles250`, `miles500`), les modes `aucune` et `dynamique` restent bloqués.

3. **Aucun mode de cassage pur ne domine.** Les modes qui ne s'appuient pas sur une clique résolvent chacun 6 instances sur 15, mais pas les mêmes : `dynamique` résout `david` (232 nœuds avec FC) que `sommet` ne résout pas ; `sommet` résout `miles250` (1 957 nœuds avec FC) que `dynamique` ne résout pas. Quand les deux terminent, `dynamique` est meilleur (`myciel4` : 1 293 contre 5 890 nœuds ; `queen6_6` : 254 contre 33 266).

4. **Les mécanismes se complètent sur les graphes à petite clique.** Sur `myciel4`, `clique+dyn` fait mieux que chacun des deux : fixer deux sommets (la clique) laisse la règle dynamique agir sur les 21 autres (984 nœuds contre 1 964 pour `clique` seule et 1 293 pour `dynamique` seule ; MAC : 329 contre 654 et 421). Sur toutes les autres instances, `clique+dyn` se comporte comme `clique`.

5. **Trois instances restent hors de portée.** `myciel5`, `myciel6` et `queen8_8` ne sont résolues par aucun mode en 3 s : leur clique ($\omega = 2$ pour les graphes de Mycielski, qui n'ont pas de triangle ; $\omega = 8$ pour `queen8_8` avec $\chi = 9$) ne donne aucune borne utile, et la symétrie n'est pas le seul obstacle.

6.5 Résultats : recherche d'une solution ($k = \chi$)

Pour 12 instances sur 15, le problème faisable à $k = \chi$ se résout *sans aucun retour arrière*, quel que soit le mode : le nombre de nœuds est égal à $|V|$ (par exemple 138 pour `anna`, 128 pour `miles500`). Dans ce cas le cassage de symétrie ne peut rien apporter. Seules `queen6_6`, `queen7_7` et `queen8_8` demandent de la recherche (tableau 6) ; `queen8_8` n'est résolue par aucun mode en 3 s, même à $k = \chi = 9$.

Instance, k	Conf.	`aucune`	`sommet`	`clique`	`dynamique`	`clique+dyn`
`queen6_6`, k=7	FC	1 459	3 975	2 818	1 459	2 818
	MAC	497	1 292	823	497	823
`queen7_7`, k=7	FC	76	193	3 329	76	3 329
	MAC	53	81	1 064	53	1 064

Tab. 6 : Instances faisables nécessitant de la recherche : nombre de nœuds.

Sur ces instances (tableau 6), *le cassage ne sert à rien et peut nuire*. Imposer les couleurs d'une clique tirée sans rapport avec l'heuristique de branchement dévie la recherche d'un coloriage que `dom` trouvait directement : 76 → 3 329 nœuds avec FC et 53 → 1 064 avec MAC sur `queen7_7`. C'est cohérent avec le rôle des symétries : elles ne gênent que lorsqu'il faut *prouver* qu'il n'y a pas de solution. La règle dynamique, elle, est neutre en nœuds (mêmes valeurs que `aucune`). La dichotomie complète du tableau 3 confirme l'effet sur `queen7_7` : 1 095 nœuds sans cassage contre 14 662 avec `clique+dyn` (FC).

Surcoût par nœud de la règle dynamique. Le calcul des signatures (algorithme 5) coûte $O(|V| \cdot |L|)$ par nœud, et ce surcoût croît donc avec la taille du graphe. Sur `queen6_6` ($|V| = 36$, $k = 7$, FC), 1 459 nœuds prennent 0,028 s avec `dynamique` contre 0,024 s sans : le surcoût est faible (facteur 1,2). Sur des graphes plus grands, où les 74 à 138 nœuds de la recherche se résolvent en quelques millisecondes, il se voit nettement : `anna` ($|V| = 138$, FC) : 0,022 s contre 0,005 s (facteur 4,4) ; `huck` ($|V| = 74$) : 0,011 s contre 0,002 s (facteur 5,5) ; `miles500` ($|V| = 128$) : 0,034 s contre 0,009 s (facteur 3,8). Les temps étant très petits, ce sont des ordres de grandeur, et il y a des exceptions (sur `games120`, $|V| = 120$, les deux temps sont voisins : 0,031 s contre 0,034 s). La tendance générale reste que la règle dynamique est peu coûteuse sur de petits graphes et coûte plusieurs fois plus par nœud sur des graphes de 70 sommets ou plus.

6.6 Conséquence pour la dichotomie

La dichotomie (algorithme 4) a besoin de prouver l'infaisabilité d'un $k < \chi$ *seulement si* la borne inférieure lb obtenue par la clique est inférieure à χ. Les résultats précédents conduisent donc à distinguer deux cas.

- $\omega = \chi$ (9 instances sur 15). La borne $lb = \omega$ est exacte dès le départ : tous les appels de décision sont faisables, la dichotomie n'a *jamais* à prouver d'infaisabilité, et le cassage de symétrie est inutile ou, comme sur `queen7_7`, nuisible.
- $\omega < \chi$ (`myciel3` à `myciel6`, `queen6_6`, `queen8_8`). Il faut prouver l'infaisabilité pour $k = \chi - 1$, et c'est là que le cassage de symétrie est décisif : il permet de conclure sur `myciel3`, `myciel4` et `queen6_6` (où le modèle de base échoue avec MAC dans la dichotomie du tableau 3), mais ne suffit pas pour `myciel5`, `myciel6` et `queen8_8`.

Cela suggère de ne fixer la clique que lorsque $\omega < \chi$ (appels dont on s'attend à ce qu'ils soient infaisables), et de ne pas la fixer pour les appels faisables. Dans tous les cas, *calculer* la clique

pour la borne inférieure ne coûte presque rien.

6.7 Limites de l'étude

- **Instances difficiles.** Sur `myciel5`, `myciel6` et `queen8_8`, prouver l'infaisabilité à $k = \chi - 1$ reste hors de portée en 3 s pour tous les modes ; pour `queen8_8`, même trouver un 9-coloriage échoue en 3 s. Les symétries ne sont donc pas l'unique obstacle ; il faudrait de meilleures bornes inférieures (les graphes de Mycielski n'ont pas de triangle, donc la clique vaut 2), de meilleures heuristiques de recherche, ou de l'apprentissage de nogoods (cours 3).
- **Un seul réglage de recherche.** Nous avons fixé l'heuristique à `dom` pour isoler l'effet du cassage ; l'interaction avec d'autres heuristiques (par exemple choisir en priorité les sommets de la clique) n'est étudiée que ponctuellement avec `dom_wdeg` (tableau 3).
- **Temps courts.** Les temps sont de l'ordre de la milliseconde ou de la seconde ; on a surtout comparé les nombres de nœuds, qui sont indépendants de la machine, et les temps en ordre de grandeur. Les temps varient d'une exécution à l'autre selon la charge de la machine.
- **Timeouts.** Quand une exécution atteint le timeout, son nombre de nœuds est celui du temps imparti, non comparable à celui d'une exécution terminée : un TO signifie « non résolu en 3 s » (ou 10 s pour le tableau 3) et non « plus long que les autres ». Un timeout plus long peut faire terminer certaines exécutions, notamment sans cassage de symétrie : c'est le cas de `queen6_6` avec FC, résolue sans cassage en 181 975 nœuds avec un timeout de 10 s.
- **Cassage incomplet.** Aucune de ces méthodes n'élimine *toutes* les symétries (il en reste, par exemple, entre sommets de même voisinage ou provenant des automorphismes du graphe).

6.8 Conclusion de l'approfondissement

1. Là où les symétries de couleurs sont le vrai obstacle (graphes de Mycielski de petite taille, `queen6_6`), les casser réduit l'arbre d'un à plus de deux ordres de grandeur et fait passer des problèmes du statut « non résolu » à « résolu ».
2. Sur les instances DIMACS à grosse clique ($\omega = \chi$, 9 instances sur 15), le facteur décisif est la *borne inférieure* donnée par la clique : elle prouve l'infaisabilité sans exploration, alors que le seul cassage de symétrie (`sommet` ou `dynamique`) échoue sur 6 d'entre elles. Distinguer ces deux effets est essentiel pour interpréter les gains.
3. Imposer une clique peut dégrader la recherche d'une solution (`queen7_7` : 76 → 3 329 nœuds ; 1 095 → 14 662 dans la dichotomie complète) ; la règle dynamique est neutre en nœuds mais son surcoût par nœud croît avec la taille du graphe (d'un facteur voisin de 1 à plus de 5 dans nos essais).
4. Recommandation : toujours calculer la clique pour la borne inférieure ; casser les symétries (`clique+dyn`) pour les appels infaisables quand $\omega < \chi$; ne pas les casser pour les appels faisables.
5. Réduire le nombre de nœuds ne signifie pas toujours réduire le temps : il faut comparer le coût du raisonnement ajouté et le parcours évité, comme pour FC contre MAC dans les n-Reines.

7 Bilan

7.1 Points forts

- Un solveur réellement *générique* : le même code résout les n-Reines et le coloriage ; les problèmes sont des fonctions qui construisent un `CSP`, sans lien avec le moteur.
- Des stratégies indépendantes et activables à la demande (consistance, heuristiques, symétries, timeout), ce qui permet des études expérimentales propres.
- Une validation par comparaison croisée : toutes les stratégies doivent trouver les mêmes solutions, et un vérificateur indépendant contrôle chaque solution.
- Une gestion des domaines par trail, sans copies, et des supports précalculés.

7.2 Points faibles

- Les contraintes en extension coûtent en mémoire (jusqu'à d^2 tuples par arête) et la construction des n-Reines est en $O(n^4)$: limitant pour les grands n.
- Python limite le débit (de l'ordre de 10^3 à 10^5 nœuds par seconde selon le niveau de consistance dans nos essais) : les instances DIMACS les plus grosses restent hors de portée.
- Seul AC3 est implémenté (pas AC4 ni AC6) ; pas de techniques de *look-back* (backjumping, learning) ; pas de contraintes globales (`alldifferent`) ni de contraintes n-aires.
- Pour le coloriage, la borne inférieure par clique gloutonne est faible (elle vaut 2 sur les graphes sans triangle), ce qui limite la dichotomie ; trois instances (`myciel5`, `myciel6`, `queen8_8`) ne sont pas résolues.

7.3 Perspectives

Dans l'ordre de l'effort croissant : (i) améliorer la borne inférieure du coloriage (clique exacte, bornes basées sur les cliques maximales) ; (ii) implémenter AC4 et comparer à AC3 ; (iii) ajouter du backjumping dirigé par les conflits et un apprentissage de nogoods ; (iv) ajouter des contraintes globales (`alldifferent`) pour des modèles plus compacts ; (v) étudier des heuristiques qui privilégient les sommets de la clique imposée.

8 Conclusion

Nous avons réalisé un mini-solveur de CSP binaires à variables entières, avec backtrack, forward-checking, MAC et des heuristiques interchangeables, validé par comparaison croisée de ses stratégies. Les benchmarks sur les n-Reines retrouvent les enseignements du cours : l'heuristique de variable pèse plus que la consistance, et diminuer le nombre de nœuds ne diminue pas toujours le temps. L'étude des symétries du coloriage, menée sur 15 instances DIMACS, montre que les couleurs interchangeables sont un obstacle réel pour prouver l'infaisabilité quand la clique ne donne pas une borne suffisante (graphes de Mycielski, `queen6_6`) : le cassage de symétrie y réduit l'arbre d'un à plus de deux ordres de grandeur. Sur les autres instances, la borne inférieure par clique suffit, et le cassage de symétrie est inutile, voire nuisible pour la recherche d'une solution. L'heuristique `dom_wdeg` réduit en outre le nombre de nœuds de MAC d'un facteur 2 à 7 sur les instances de coloriage avec cassage de symétrie. Trois instances (`myciel5`, `myciel6`, `queen8_8`) restent non résolues et appellent de meilleures bornes inférieures ou de l'apprentissage.

A Utilisation et reproduction

Exemple d'utilisation.

```
from problemes import reines, coloriage, lire_col, chromatique
from solveur import Solveur

# n-Reines avec MAC et l'heuristique "plus petit domaine"
csp = reines(20)
s = Solveur(csp, consistance="mac", heuristique_var="dom", timeout=10)
solution = s.resoudre() # dict {variable: valeur} ou None
print(s.stats) # noeuds, retours, revisions, temps, timeout

# Nombre chromatique d'une instance DIMACS
n, aretes = lire_col("instances/myciel4.col")
r = chromatique(n, aretes, timeout=10, symetrie="clique+dyn",
              consistance="mac", heuristique_var="dom")
print(r["lb"], r["ub"], r["prouve"])
```

Commandes.

```
python tests.py # validation croisee
python benchmark.py instances myciel3 myciel4 # n-Reines + coloriage
python benchmark.py instances myciel3 myciel4 queen5_5 queen6_6 queen7_7 \
      --sans-reines --symetrie=aucune # dichotomie sans casser les symetries
python etude_symetries.py instances 3 # etude des symetries (timeout 3 s)
```

B Description des fichiers

Fichier	Contenu
`csp.py`	classe `CSP` : variables, domaines, contraintes binaires en extension (symétrisées), supports précalculés, vérificateur de solution
`solveur.py`	classe `Solveur` : backtrack, trail, forward-checking, AC3/MAC, heuristiques de variable et de valeur, timeout, statistiques
`problemes.py`	n-Reines, coloriage (5 modes de symétrie), lecteur `.col`, clique et coloriage gloutons, dichotomie du nombre chromatique
`benchmark.py`	benchmarks n-Reines et coloriage (options `--sans-reines`, `--symetrie=`…)
`etude_symetries.py`	étude de l'approfondissement ; écrit `resultats_symetries.csv`
`tests.py`	tests de correction par comparaison croisée

Tab. 7 : Organisation des sources.